

Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной
математики

Кафедра вычислительной математики и программирования

Курсовой проект по курсу «Дискретный анализ»

Студент: Д. О. Кочев
Преподаватель: С. А. Сорокин
Группа: М8О-306Б-22
Дата:
Оценка:
Подпись:

Москва, 2024

Курсовой проект 24/25

Задача: Вам дан набор горизонтальных отрезков, и набор точек. Для каждой точки определите сколько отрезков лежит строго над ней. Ваше решение должно работать online, то есть должно обрабатывать запросы по одному после построения необходимой структуры данных по входным данным. Чтение входных данных и запросов вместе и построение по ним общей структуры запрещено.

Формат ввода

В первой строке вам даны два числа n и m ($1 \leq n, m \leq 10^5$) — количество отрезков и количество точек соответственно. В следующих n строках вам заданы отрезки, в виде троек чисел l, r и h ($-10^9 \leq l < r \leq 10^9, -10^9 \leq h \leq 10^9$) — координаты x левой и правой границ отрезка и координата y отрезка соответственно. В следующих m строках вам даны пары чисел x, y ($-10^9 \leq x, y \leq 10^9$) — координаты точек.

Формат вывода

Для каждой точки выведите количество отрезков над ней.

1 Описание

Для решения данной задачи я использовал персистентное Декартово дерево, а также метод сканирующей прямой. Для реализации Декартова дерева я решил использовать метод клонирования путей, так как для решения будет достаточно частичной персистентности. Пользуясь информацией из [1], мы убеждаемся, что все операции в нем будут выполняться за $\Theta(\log n)$, где n - количество вершин.

Чтобы использовать метод сканирующей прямой, мы создадим вектор *arr_segment* левых и правых x отрезков, отсортируем. Далее пройдемся по данному вектору и будем добавлять или удалять отрезки из дерева, когда x - левая граница отрезка или правая соответственно. При этом при каждой операции будем создавать новую версию и хранить в узле координату y .

Далее для получения ответа будем идти по вектору *arr_segment*, и останавливаться, когда координата x станет меньше i -го элемента. Тогда нам надо обратиться в $(i - 1)$ -ую версию нашего дерева, и подсчитать все узлы, в которых значения будут больше координаты y введенной точки.

Это число и будет являться ответом на вопрос, сколько отрезков находятся над введенной точкой. Подсчет узлов, в которых значение больше координаты y , имеет сложность $\Theta(\log n)$.

2 Исходный код

Мой код можно логически поделить на две части. Сначала я реализую персистентное Декартово дерево.

```
1 struct Node {
2     int value;
3     int priority;
4     int size;
5     Node* left;
6     Node* right;
7
8     Node(int val, int pr) : value(val), priority(pr), size(1), left(nullptr), right(
9         nullptr) {}
10 };
11
```

Важным элементом в коде дерева является реализация персистентности в вставке и удалении.

```
1 Node* insert(Node* root, int value, int priority) {
2     if (!root) {
3         return new Node(value, priority);
4     }
5     Node* newNode = new Node(*root); //clone node
6     if (value < root->value) {
7         newNode->left = insert(root->left, value, priority);
8         if (newNode->left && newNode->left->priority > newNode->priority) {
9             newNode = rotateRight(newNode);
10        }
11    } else {
12        newNode->right = insert(root->right, value, priority);
13        if (newNode->right && newNode->right->priority > newNode->priority) {
14            newNode = rotateLeft(newNode);
15        }
16    }
17    updateSize(newNode);
18    return newNode;
19 }
```

Во второй части кода я реализую метод сканирующей прямой, поиск нужной версии дерева и вывод ответа.

```
1 int main() {
2     int n, m;
3     std::vector<Segment> arr_segment;
4     std::cin >> n >> m;
5     int l, r, h;
6     for (int i = 0; i < n; i++) {
```

```

7         std::cin >> l >> r >> h;
8         arr_segment.push_back({l, h, 1}); //create node
9         arr_segment.push_back({r, h, 0}); //delete node
10    }
11    std::sort(arr_segment.begin(), arr_segment.end(),
12    [](const Segment& a, const Segment& b) {
13        if (a.x != b.x) {
14            return a.x < b.x;
15        }
16        return a.isCreate > b.isCreate;
17    });
18    Node* root = nullptr;
19    std::vector<Node*> versions;
20    for (int i = 0; i < arr_segment.size(); i++) {
21        if (arr_segment[i].isCreate == 1) {
22            root = insert(root, arr_segment[i].y, i);
23            versions.push_back(root);
24        } else {
25            root = remove(root, arr_segment[i].y);
26            versions.push_back(root);
27        }
28    }
29    std::vector<Point> arr_points;
30    int x, y;
31    for (int i = 0; i < m; i++) {
32        std::cin >> x >> y;
33        arr_points.push_back({x, y});
34    }
35    for (int i = 0; i < m; i++) {
36        for (int j = 0; j < arr_segment.size(); j++) {
37            if (arr_points[i].x <= arr_segment[j].x) {
38                if (arr_segment[j].isCreate == 1 && (arr_points[i].x == arr_segment[j].x
39                    )) continue;
40                if (j == 0){
41                    std::cout<<0<<std::endl;
42                }
43                else{
44                    int count = segmentsAbovePoints(arr_points[i].y, versions[j - 1]);
45                    std::cout << count << std::endl;
46                }
47                break;
48            }
49            else if (j == arr_segment.size() - 1){
50                std::cout<<0<<std::endl;
51            }
52        }
53    }
54    return 0;

```

kr.cpp	
void updateSize(Node* node)	Функция сохранения высоты поддерева
Node* rotateRight(Node* y)	Правый поворот
Node* rotateLeft(Node* x)	Левый поворота
Node* rotateRightCopy(Node* y)	Правый поворот с копированием узла
Node* rotateLeftCopy(Node* x)	Левый поворот с копированием узла
Node* insert(Node* root, int value, int priority)	Вставка в персистентное дерево
Node* remove(Node* root, int value)	Удаление из персистентного дерева
int segmentsAbovePoints(int y, Node* root)	Функция поиска узлов, которые больше передаваемого y
Node* remove(Node* root, int value)	Функция удаления из персистентного дерева

3 Консоль

```
dmitrykochev$ cat test1.txt
5 3
0 5 2
1 3 1
1 8 5
-2 0 3
-1 6 -1
1 0
3 -3
3 5
dmitrykochev$ ./kp <test1.txt
3
4
0
dmitrykochev$ cat test2.txt
3 4
0 4 3
4 7 2
1 8 1
0 0
1 -1
4 0
8 -2
dmitrykochev$ ./kp <test2.txt
1
2
3
1
```

4 Тест производительности

В тесте производительности я замеряю, сколько по времени работает моя программа, а также сколько памяти она занимает при разном количестве данных. Первый тест будет содержать 100 отрезков и 100 точек для проверки. Второй содержит 10^4 отрезков, 10^4 точек. Третий - 10^5 отрезков, 10^5 точек.

```
dmitrykochev$ ./benchmark <test1.txt
Algorithm time: 6951us
Algorithm use: 69648 bytes
dmitrykochev$ ./benchmark <test2.txt
Algorithm time: 899853us
Algorithm use: 14875008 bytes
dmitrykochev$ ./benchmark <test3.txt
Algorithm time: 14239335us
Algorithm use: 193240640 bytes
```

Последний тест занимает примерно 14 секунд и 184 мегабайта. Это укладывается в ограничения задачи. Также это доказывает эффективность использования персистентного дерева и метода сканирующей прямой.

5 Выводы

Выполнив курсовой проект по предмету «Дискретный анализ», я научился решать сложную задачу, связанную с определением места нахождения точки относительно нескольких отрезков. Безусловно, данный курсовой проект дал мне ценнейший опыт в написании и оптимизации кода. Мне пришлось продумывать реализацию и работу сложной структуры данных, а так же замерять количество памяти, которую она использует. Все это, определенно, поможет мне в решении больших задач в будущем.

Список литературы

[1] *Персистентные структуры данных*

URL: https://neerc.ifmo.ru/wiki/index.php?title=Персистентные_структуры_данных (дата обращения: 11.11.2024).